

Runbook — PDPL data residency: move to me-central2 (Dammam)

SECTION 06-operations-it TYPE runbook STATUS **active** OWNER Founder UPDATED 2026-06-16

Status: Web app + AI backend LIVE on `flygaca-app` — Firestore in Dammam; data + Auth + hosting + the `chat` gateway + Captain Adel service all deployed and verified. Compute runs **interim in me-central1 (Doha)** (me-central2 Cloud Run not yet granted). Remaining: billing secrets, OAuth providers, custom-domain DNS, iOS, and the me-central2 region migration. **Created:** 2026-06-01

Project history (important): the first attempt provisioned `flygaca-ksa`; it was later deleted and the work redone on `flygaca-app` (the keeper). The original `flygaca-firebase` was also deleted. Both deletions were intentional. Because `flygaca-firebase` is gone, its `GEMINI_API_KEY` could not be recovered and must be re-supplied (see step 3a). The data + Auth survive only in the local backup `_pdpl_migration_backup/` (restored into `flygaca-app`).

Compute region constraint (me-central2 not yet available)

`gcloud run deploy ... --region me-central2` on this account fails with `LOCATION_POLICY_VIOLATED` / "contact sales" — it is NOT an org policy (`gcp.resourceLocations` = `ALLOW`), it's an account-level regional allowlist only Google can lift. Cloud Functions v2 run on Cloud Run, so this blocks ALL compute in Dammam, not just the Adel service. **Decision:** run compute in me-central1 (Doha) on an interim basis and request me-central2 access; migrate compute to Dammam once granted. Firestore me-central2 is unaffected (generally available).

- The region is config-driven: `functions/region.js` defaults to me-central2 but `functions/.env.flygaca-app` pins `FUNCTIONS_REGION=me-central1` for now. To migrate: set it to me-central2 (or delete the line), flip the two `firebase.json` rewrites + `assets/js/billing.js`, and redeploy.
- **Action for you:** request Cloud Run access in me-central2 for `flygaca-app` via Google Cloud support/sales (only the account holder can).

Completed on flygaca-app (2026-06-01)

- ☒ Firestore `(default)` created in **me-central2 (Dammam)** — verified.
- ☒ **8 docs restored** from the local backup (3 waitlist + 5 users); rules deployed.
- ☒ **5 Auth users imported** with UIDs + passwords preserved (3 Google + 2 email/ password); Identity Platform initialized, email/password enabled, authorized domains set (`flygaca-app.web.app`, `.firebaseapp.com`, `flygaca.com`, `www`).

- ☒ **Hosting deployed** → `https://flygaca-app.web.app` (HTTP 200).
- ☒ Shared `ADEL_API_KEY` minted in Secret Manager; runtime SA granted `secretAccessor`.
- ☒ `GEMINI_API_KEY` **recovered** (brief undelete of flygaca-firebase → copied → re-deleted) and stored in flygaca-app Secret Manager.
- ☒ **Captain Adel service deployed** to Cloud Run me-central1: `https://captadel-30479965011.me-central1.run.app` — health 200, `/v1/chat` returns answers with RAG sources.
- ☒ `chat` **gateway deployed** to me-central1 (`region.js` hardcodes the interim region — a `process.env` read evaluates before firebase loads `.env`). Verified end-to-end: `https://flygaca-app.web.app/api/chat` → chat → Adel → Gemini+RAG.
- ☒ Created **placeholder** Secret Manager entries for the 5 billing secrets (`STRIPE_*`, `REVENUECAT_WEBHOOK_AUTH`) so deploy analysis passes — value is `PLACEHOLDER-...`; **replace with real values before deploying billing functions.**

Still blocked / needs you: (2) OAuth providers Google/Apple/GitHub/Twitter (email/password already works); (3) real Stripe + RevenueCat secret values, then deploy `--only functions` (billing); (5) webhook URLs; (8) custom-domain DNS flip (`flygaca.com` → flygaca-app); (9) iOS `GoogleService-Info.plist`; request **me-central2 Cloud Run access** then flip `region.js` to move compute in-Kingdom.

Why this exists

The original project `flygaca-firebase` had its Cloud Firestore (`default`) database created in `nam5` (**US multi-region**) — not in the Kingdom. A Firestore location is **permanent and cannot be changed**, so the database could not be relocated in place. Meanwhile [privacy.html](#) §4 tells the public that personal data "is stored within the Kingdom of Saudi Arabia ... in the me-central2 (Dammam) region." That statement was **not true** of the live system, which is both a PDPL data-residency problem and an inaccurate privacy notice.

Cloud Functions also ran in `me-central1` (**Doha, Qatar**) — outside the Kingdom, so personal-data *processing* was also a cross-border activity under PDPL.

Remedy: a fresh project, `flygaca-app`, with Firestore *and* Cloud Functions both in `me-central2` (**Dammam, KSA**). Pre-launch and only 8 personal records, so this is the cheapest time to fix it cleanly.

New project facts

	Value
Project ID	<code>flygaca-app</code> (project number <code>30479965011</code>)
Billing	<code>010F96-1D57E6-773AF0</code> (same account as the old project)
Firestore (<code>default</code>)	me-central2 (Dammam) , Native mode — confirmed

	Value
Cloud Functions region	target me-central2 ; interim me-central1 (Doha) until Cloud Run access granted (config-driven via <code>functions/region.js</code> + <code>.env.flygaca-app</code>)
Adel service (Cloud Run)	deployed <code>https://captadel-30479965011.me-central1.run.app</code> (me-central1, interim)
<code>chat</code> gateway	deployed me-central1 → <code>https://flygaca-app.web.app/api/chat</code> (verified)
Web app ID	<code>1:30479965011:web:7764f29e44c7e365b91fa6</code>
Hosting domains	<code>flygaca-app.web.app</code> , <code>flygaca-app.firebaseio.com</code>

Already done (by the migration so far)

- ☒ Created `flygaca-app` , linked billing, enabled APIs.
- ☒ Created Firestore (default) in **me-central2**.
- ☒ Registered the web app; captured config into [assets/js/firebase-config.js](#).
- ☒ Repointed `.firebaseerc` (default → `flygaca-app` ; old kept as alias `legacy-us`).
- ☒ Made the Cloud Functions `region` config-driven via [functions/region.js](#) (default me-central2; interim me-central1 via `.env.flygaca-app`); set the hosting rewrites in [firebase.json](#) + client region in [assets/js/billing.js](#) to me-central1; CORS allowlists updated in `functions/index.js` + `functions/content.js` .
- ☒ Deployed Firestore security rules to `flygaca-app` .
- ☒ Migrated the 8 personal-data documents (3 waitlist + 5 users) into the Dammam DB. Tools: [scripts/migrate-firestore.js](#) (live source→dest) and [scripts/restore-firestore-from-backup.js](#) (from the local backup, used here since the live source was deleted).
- ☒ Backed up the live source data to `_pdp1_migration_backup/` **outside the git repo** (never committed — it is PII).

Remaining cutover steps

Most of these need secrets/credentials or the Firebase/3rd-party consoles, so they are not automated. Run them roughly in order. **Do the Auth import before re-running the data copy at cutover**, so `users/{uid}` docs line up with real Auth UIDs.

1. Migrate Authentication users (preserve UIDs)

The app uses **email/password + Google + Apple + GitHub + Twitter**.

```

firebase auth:export users-legacy.json --format=json --project flygaca-firebase
# Federated users (Google/Apple/etc.) import with no password.
# Email/password users need the SOURCE project's password-hash params:
#   Firebase console (flygaca-firebase) → Authentication → Users → :
#   → "Password hash parameters" (algo=SCRYPT, signer key, salt separator, rounds, mem
cost)
firebase auth:import users-legacy.json --project flygaca-app \
  --hash-algo=SCRYPT --hash-key=<KEY> --salt-separator=<SEP> --rounds=8 --mem-cost=14
rm users-legacy.json # contains password hashes – do not keep / commit

```

2. Re-enable Auth sign-in providers in `flygaca-app`

Console → Authentication → Sign-in method. Enable Email/Password, Google, Apple, GitHub, Twitter. Each federated provider needs its **OAuth client ID/secret** re-entered (the old project's are not reused). Add **Authorized domains**: `flygaca-app.web.app`, `flygaca-app.firebaseio.com`, and `flygaca.com` / `www.flygaca.com` once the custom domain is attached.

3. Set Cloud Functions secrets

`ADEL_API_KEY` is already minted in Secret Manager. Still needed:

```

# Billing functions (values from the Stripe + RevenueCat dashboards – not yet supplied):
for S in STRIPE_SECRET_KEY STRIPE_WEBHOOK_SECRET STRIPE_PRICE_MONTHLY \
  STRIPE_PRICE_ANNUAL REVENUECAT_WEBHOOK_AUTH; do
  firebase functions:secrets:set "$S" --project flygaca-app
done

```

`ADEL_API_URL` (the Adel service base URL) is a non-secret param, already set in [functions/.env.flygaca-app](#) once 3a is done. Optional env toggles (see [functions/README.md](#)):

`ADEL_PROTECTED_CONTENT`, `ADEL_RC_REQUIRE_BINDING`, `ADEL_APPCHECK_MODE`.

3a. Supply GEMINI_API_KEY + deploy the Captain Adel service (Cloud Run)

`GEMINI_API_KEY` was lost with the deleted `flygaca-firebase` and MUST be re-supplied:

```

printf '%s' "<gemini-key>" | gcloud secrets create GEMINI_API_KEY \
  --project flygaca-app --replication-policy=automatic --data-file=-
# Deploy the Adel brain (interim me-central1 until me-central2 access):
gcloud run deploy captadel --source captadel/ --region me-central1 \
  --project flygaca-app --memory 2Gi --cpu 2 --port 8787 --allow-unauthenticated \
  --set-env-vars MODEL_PROVIDER=gemini,CAPTAIN_ADEL_MODEL=gemini-2.5-flash \
  --set-secrets GEMINI_API_KEY=GEMINI_API_KEY:latest,ADEL_API_KEY=ADEL_API_KEY:latest \
  --max-instances 4
# Put the resulting URL into functions/.env.flygaca-app ADEL_API_URL.

```

PDPL note: real user questions are personal data. me-central1 (Doha) is outside the Kingdom — this is the documented interim gap to clear once me-central2 Cloud Run is granted. Per [phase0.md](#) the service must store no PII.

4. Deploy functions to `flygaca-app` (interim me-central1)

```
firebase deploy --only functions:chat --project flygaca-app # AI gateway (needs 3a)
# Billing/other functions once their secrets exist:
# firebase deploy --only functions --project flygaca-app
```

Functions deploy to me-central1 (per `.env.flygaca-app`). Note the resulting `chat`, `stripeWebhook`, `revenuecatWebhook` URLs for the next step.

5. Update Stripe + RevenueCat webhooks

- **Stripe** dashboard → Developers → Webhooks: point the endpoint at the new `stripeWebhook` URL; copy the new signing secret into `STRIPE_WEBHOOK_SECRET` (re-run step 3 for that one, then redeploy).
- **RevenueCat** dashboard → integrations/webhooks: point at the new `revenuecatWebhook` URL; reset `REVENUECAT_WEBHOOK_AUTH` to match.

6. App Check — DONE for web (monitor mode)

✓ Wired: reCAPTCHA Enterprise key `6LeLugct...015P` registered for the web app via the App Check Admin API; client init in [assets/js/firebase-init.js](#) (used by auth/waitlist/schools + chat); CSP widened for `www.google.com`; the chat function runs `ADEL_APPCHECK_MODE=monitor` (logs token validity, never blocks — verified a token-less `/api/chat` still answers). **To finish:** (a) confirm the reCAPTCHA key's allowed domains include `flygaca-app.web.app` / `flygaca.com` / `localhost`; (b) watch the chat function logs for valid tokens from real browsers; (c) once clean, set `ADEL_APPCHECK_MODE=enforce` and (optionally) enable App Check enforcement for Firestore/Functions in the console. iOS still needs DeviceCheck/App Attest.

7. Deploy hosting + verify on the temporary domain

```
firebase deploy --only hosting --project flygaca-app
```

Smoke-test at `https://flygaca-app.web.app`: sign in, waitlist submit, logbook write, `/api/chat`, checkout. Confirm reads/writes land in the Dammam DB.

8. Move the custom domain

Firebase console (`flygaca-app`) → Hosting → Add custom domain → `flygaca.com` (+ `www`). Update DNS at the registrar to the records Firebase shows. This is the real cutover: traffic now serves from the in-Kingdom project.

9. iOS / Capacitor

Download a fresh **GoogleService-Info.plist** for `flygaca-app`, replace it in the `ios/` project, update the reversed-client-ID URL scheme, and rebuild (see [office/runbook-ios.md](#)). Native Google/Apple sign-in uses the new project's OAuth config.

10. Final data sync at cutover

Immediately before flipping DNS (and after Auth import), re-run the copy to catch any signups that arrived in the meantime:

```
node scripts/migrate-firestore.js --dry-run    # review
node scripts/migrate-firestore.js              # apply
```

11. Decommission the legacy US project

Once `flygaca.com` serves from `flygaca-app` and is verified for a safe window:

- Disable the old waitlist/write paths (or just rely on DNS already being moved).
- Delete the personal data in the old project, then the project itself:

```
# confirm nothing still points at it, then:
gcloud projects delete flygaca-firebase
```

(Deleting the project is the cleanest way to erase the nam5 personal data. Keep the local backup until you are certain the new project is healthy.)

- Also delete the stray empty project `fly-gaca-495116` (noted in PHASE0 P0-5).

12. Privacy policy + docs

- The [privacy.html](#) §4 claim (data in me-central2/Dammam) becomes **true only after step 8 (DNS cutover)** — until then the live site is still served from the nam5 project. Do not treat the claim as satisfied early.
- PHASE0 P0-5 has been corrected to record the real region history.

Rollback

Until step 8 (DNS), the live site is untouched on `flygaca-firebase`; abandoning the migration costs nothing but the new (cheap, idle) project. After DNS cutover, rollback = repoint DNS back to the old project (its data is intact until step 11). Do **not** run step 11 until you are confident.